Lecture 5. Import and Tidy Data

Tibbles, readr parsing, tidy data, pivoting, separating and uniting

Contents

5.1 Tibbles **2**

5.2 Data import with readr **3**

5.2.1 Parsing a vector 3

5.2.2 Parsing a file 5

5.2.3 Writing files and other formats 5

5.3 Tidy data **5**

5.4 Gathering and spreading **7**

5.5 Separating and uniting **8**

5.1 Tibbles

Motivation. Tibbles are data frames with a few old behaviours removed.

A **tibble** never changes input types (no strings to factors) or names, never creates row names, and complains about missing variables.

```
as_tibble(iris)                          # data frame -> tibble
```

```
## # A tibble: 150 x 5
##   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
##          <dbl>       <dbl>        <dbl>       <dbl> <fct>
## 1          5.1         3.5          1.4         0.2 setosa
## 2          4.9         3            1.4         0.2 setosa
## 3          4.7         3.2          1.3         0.2 setosa
## 4          4.6         3.1          1.5         0.2 setosa
## 5          5           3.6          1.4         0.2 setosa
## # i 145 more rows
```

```
tibble(x = 1:5, y = 1, z = x ^ 2 + y)  # from vectors; y is recycled
```

```
## # A tibble: 5 x 3
##       x     y     z
##   <int> <dbl> <dbl>
## 1     1     1     2
## 2     2     1     5
## 3     3     1    10
## 4     4     1    17
## 5     5     1    26
```

```
tribble(                                 # "transposed tibble": written row by row
  ~x, ~y, ~z,
  "a", 2, 3.6,
  "b", 1, 8.5
)
```

```
## # A tibble: 2 x 3
##   x         y     z
##   <chr> <dbl> <dbl>
## 1 a         2   3.6
## 2 b         1   8.5
```

`as.data.frame()` converts back. Non-syntactic names (`':)'`, `'2000'`) need backticks.

Printing. First 10 rows, columns that fit, with types. More: `print(n = 10, width = Inf)`, `options(tibble.print_max = n, tibble.print_min = m)` (over `n` rows, print `m`), `options(tibble.width = Inf)`.

Subsetting. `$` by name, `[[ ]]` by name or position; no partial matching, a warning for missing columns. In a pipe `.` is the incoming object:

```
df <- tibble(x = runif(3), y = rnorm(3))
df$x
```

```
## [1] 0.08075014 0.83433304 0.60076089
```

```
df[[2]]
```

```
## [1] -1.00599730 -0.08433889 -0.55406480
```

```
df %>% .$x
```

```
## [1] 0.08075014 0.83433304 0.60076089
```

5.2 Data import with readr

Motivation. Turning characters into numbers, dates and factors is guesswork: how readr guesses, and how to correct it.

`readr` turns flat files into tibbles:

- `read_csv()` (comma), `read_csv2()` (semicolon; comma decimal mark), `read_tsv()` (tab), `read_delim()` (any);
- `read_fwf()` (fixed width), `read_table()` (white space), `read_log()` (Apache logs).

Shared arguments. First argument a path (`read_csv("data/heights.csv")`) or an inline string:

```
read_csv("a,b,c
1,2,3
4,5,6")
```

```
## # A tibble: 2 x 3
##       a     b     c
##   <dbl> <dbl> <dbl>
## 1     1     2     3
## 2     4     5     6
```

Useful arguments:

```
read_csv("The first line of metadata
  The second line of metadata
  x,y,z
  1,2,3", skip = 2)                                  # skip n lines
read_csv("# A comment I want to skip
  x,y,z
  1,2,3", comment = "#")                             # drop comment lines
read_csv("1,2,3\n4,5,6", col_names = FALSE)          # no header: X1, X2, X3
read_csv("1,2,3\n4,5,6", col_names = c("x", "y", "z"))
read_csv("a,b,c\n1,2,.", na = ".")                   # symbol for missing values
```

Versus `read.csv()`: faster, tibbles, no factors, same on every OS.

5.2.1 Parsing a vector

`parse_*()` turn character vectors into specialised ones (building blocks of `read_csv()`).

```
str(parse_logical(c("TRUE", "FALSE", "NA")))
```

```
##  logi [1:3] TRUE FALSE NA
```

```
str(parse_date(c("2010-01-01", "1979-10-14")))
```

```
##  Date[1:2], format: "2010-01-01" "1979-10-14"
```

```
parse_integer(c("1", "231", ".", "456"), na = ".")
```

```
## [1]   1 231  NA 456
```

```
x <- parse_integer(c("123", "345", "abc", "123.45"))
```

```
## Warning: 2 parsing failures.
## row col               expected actual
##   3  -- no trailing characters abc
##   4  -- no trailing characters 123.45
```

```
x
```

```
## [1] 123 345  NA  NA
## attr(,"problems")
## # A tibble: 2 x 4
##     row   col expected               actual
##   <int> <int> <chr>                  <chr>
## 1     3    NA no trailing characters abc
## 2     4    NA no trailing characters 123.45
```

```
problems(x)
```

```
## # A tibble: 2 x 4
##     row   col expected               actual
##   <int> <int> <chr>                  <chr>
## 1     3    NA no trailing characters abc
## 2     4    NA no trailing characters 123.45
```

Failures become `NA` with a warning; `problems()` lists them.

Numbers. Decimal marks differ (`.`, `,`); context (`"$1000"`, `"10%"`); grouping marks (`1,000,000`, `1.000.000`, `1'000'000`).

```
parse_double("1,23", locale = locale(decimal_mark = ","))
```

```
## [1] 1.23
```

```
parse_number("It cost $123.45")
```

```
## [1] 123.45
```

```
parse_number("123.456.789", locale = locale(grouping_mark = "."))
```

```
## [1] 123456789
```

`parse_number()` ignores surrounding non-numeric characters.

Strings. Bytes (`charToRaw("Hadley")`) map to characters by an **encoding**: usually UTF-8; Latin-1 or EUC-KR (Korean) must be declared:

```
x2 <- "\xbe\xc8\xb3\xe7\xc7\xcf\xbc\xbc\xbf\xe4"     # bytes of a Korean greeting in EUC
    -KR
parse_character(x2, locale = locale(encoding = "EUC-KR"))
guess_encoding(charToRaw(x2))                        # when the encoding is unknown
```

Factors: `parse_factor(x, levels = fruit)` warns about values outside `levels`.

Dates and times. *Date*: days since 1970-01-01; *date-time*: seconds since 1970-01-01 00:00 UTC; *time*: seconds since midnight.

- `parse_datetime()`: ISO 8601, biggest to smallest component; missing time = midnight.
- `parse_date()`: year, `-` or `/`, month, `-` or `/`, day.
- `parse_time()`: hour `:` minutes, optional `:` seconds and am/pm.

```
parse_datetime("2010-10-01T2010")
```

```
## [1] "2010-10-01 20:10:00 UTC"
```

```
parse_date("2010-10-01")
```

```
## [1] "2010-10-01"
```

```
parse_time("01:10 am")
```

```
## 01:10:00
```

```
parse_date("01/02/15", "%d/%m/%y")
```

```
## [1] "2015-02-01"
```

Format codes: `%Y` (4-digit year), `%y` (2-digit), `%m`, `%b` (month abbreviation), `%d`, `%H`, `%M`, `%S`.

5.2.2 Parsing a file

readr guesses from the first 1000 rows, in order: logical (`T`, `F`, `TRUE`, `FALSE`), integer, double, number, time, date, date-time, else character. It fails if those rows are special (all missing) or `NA`s abound:

```
challenge <- read_csv(readr_example("challenge.csv"))
problems(challenge)
```

```
## # A tibble: 0 x 5
## # i 5 variables: row <int>, col <int>, expected <chr>, actual <chr>, file <chr>
```

`y` is all `NA` in the first 1000 rows, guessed logical. Specify types:

```
challenge <- read_csv(readr_example("challenge.csv"),
  col_types = cols(x = col_double(), y = col_date()))
tail(challenge)
```

```
## # A tibble: 6 x 2
##       x y
##   <dbl> <date>
## 1 0.805 2019-11-21
## 2 0.164 2018-03-29
## 3 0.472 2014-08-04
## 4 0.718 2015-08-16
## 5 0.270 2020-02-04
## # i 1 more row
```

Always give `col_types`: it documents the data and fails loudly if they change. Alternatives: `guess_max = 1001`; or read all as character (`cols(.default = col_character())`) then `type_convert()`.

5.2.3 Writing files and other formats

`write_csv()`, `write_tsv()`: UTF-8, ISO 8601 dates, types lost. `write_rds()`/`read_rds()`: binary, types kept. Packages: `haven` (SPSS, Stata, SAS), `readxl` (`.xls`, `.xlsx`), `DBI` + `RSQLite` (SQL), `jsonlite`, `xml2`.

5.3 Tidy data

Motivation. Only one layout lets the tidyverse work without fighting the data.

`table1`–`table5`: tuberculosis cases and population, three countries, 1999 and 2000, in different layouts:

```
table1
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <dbl>      <dbl>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

```
table2
```

```
## # A tibble: 12 x 4
##    country      year type           count
##    <chr>       <dbl> <chr>          <dbl>
## 1 Afghanistan  1999 cases            745
## 2 Afghanistan  1999 population 19987071
## 3 Afghanistan  2000 cases           2666
## 4 Afghanistan  2000 population 20595360
## 5 Brazil       1999 cases          37737
## # i 7 more rows
```

```
table3
```

```
## # A tibble: 6 x 3
##    country      year rate
##    <chr>       <dbl> <chr>
## 1 Afghanistan  1999 745/19987071
## 2 Afghanistan  2000 2666/20595360
## 3 Brazil       1999 37737/172006362
## 4 Brazil       2000 80488/174504898
## 5 China        1999 212258/1272915272
## # i 1 more row
```

Definition (Tidy data)

A data set is **tidy** if

1. each variable has its own column;
2. each observation has its own row;
3. each value has its own cell.

Only `table1` is tidy: `table2` spreads an observation over two rows, `table3` has two variables in a cell, `table4a`/`table4b` use `year` values as column names.

Why tidy: one set of tools, and columns suit vectorised functions.

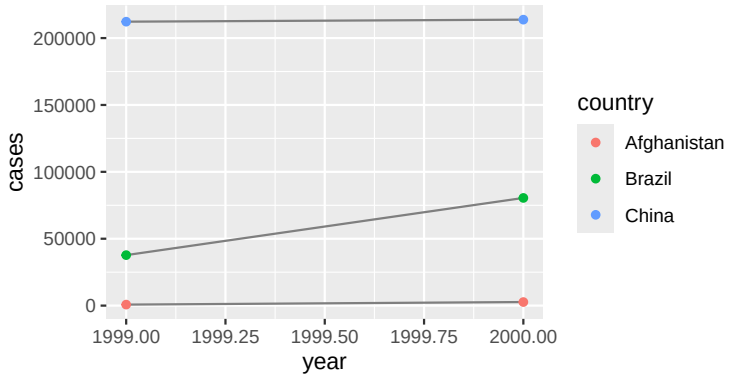
```
table1 %>% mutate(rate = cases / population * 10000)   # rate per 10,000
```

```
## # A tibble: 6 x 5
##    country      year  cases population  rate
##    <chr>       <dbl>  <dbl>      <dbl> <dbl>
## 1 Afghanistan  1999    745   19987071 0.373
## 2 Afghanistan  2000   2666   20595360 1.29
## 3 Brazil       1999  37737  172006362 2.19
## 4 Brazil       2000  80488  174504898 4.61
## 5 China        1999 212258 1272915272 1.67
## # i 1 more row
```

```
table1 %>% count(year, wt = cases)                      # cases per year
```

```
## # A tibble: 2 x 2
##    year      n
##   <dbl>  <dbl>
## 1  1999 250740
## 2  2000 296920
```

```
ggplot(table1, aes(year, cases)) +
  geom_line(aes(group = country), colour = "grey50") +
  geom_point(aes(colour = country))
```



5.4 Gathering and spreading

Motivation. A variable spread over columns, or an observation over rows: two inverse fixes.

Gathering. In `table4a` the names `1999`, `2000` are values of `year`. `gather()` makes a **key** (former column names) and a **value** (former cells).

```
table4a
```

```
## # A tibble: 3 x 3
##   country     `1999` `2000`
##   <chr>        <dbl>  <dbl>
## 1 Afghanistan    745   2666
## 2 Brazil       37737  80488
## 3 China       212258 213766
```

```
tidy4a <- table4a %>% gather(`1999`, `2000`, key = "year", value = "cases")
tidy4b <- table4b %>% gather(`1999`, `2000`, key = "year", value = "population")
left_join(tidy4a, tidy4b)
```

```
## # A tibble: 6 x 4
##   country     year   cases population
##   <chr>       <chr>  <dbl>      <dbl>
## 1 Afghanistan 1999     745   19987071
## 2 Brazil      1999   37737  172006362
## 3 China       1999  212258 1272915272
## 4 Afghanistan 2000    2666   20595360
## 5 Brazil      2000   80488  174504898
## # i 1 more row
```

`left_join()`: Lecture 6. Gathering makes tables longer.

Spreading is the inverse (wider): `spread()` turns values of `type` into columns filled from `count`.

```
spread(table2, key = type, value = count)
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <dbl>      <dbl>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

pivot_longer() and pivot_wider() [Supplement]

Current tidyr supersedes them by `pivot_longer()`, `pivot_wider()` (clearer arguments):

```
table4a %>% pivot_longer(c(`1999`, `2000`), names_to = "year", values_to = "cases")
table2 %>% pivot_wider(names_from = type, values_from = count)
```

`names_transform`, `values_drop_na` handle the type of `year` and missing cells.

5.5 Separating and uniting

`table3`'s `rate` holds two variables; `separate()` splits at any non-alphanumeric character:

```
table3 %>% separate(rate, into = c("cases", "population"), convert = TRUE)
```

```
## # A tibble: 6 x 4
##   country      year  cases population
##   <chr>       <dbl>  <int>      <int>
## 1 Afghanistan  1999    745   19987071
## 2 Afghanistan  2000   2666   20595360
## 3 Brazil       1999  37737  172006362
## 4 Brazil       2000  80488  174504898
## 5 China        1999 212258 1272915272
## # i 1 more row
```

Without `convert = TRUE` they stay character. Numeric `sep` is a position: `sep = 2` splits after character 2.

`unite()` is the inverse; `table5` has century and year split:

```
table5 %>% unite(new, century, year, sep = "")
```

```
## # A tibble: 6 x 3
##   country     new   rate
##   <chr>       <chr> <chr>
## 1 Afghanistan 1999  745/19987071
## 2 Afghanistan 2000  2666/20595360
## 3 Brazil      1999  37737/172006362
## 4 Brazil      2000  80488/174504898
## 5 China       1999  212258/1272915272
## # i 1 more row
```

(default separator `_`).